



CIBERSEGURIDAD EN ENTORNOS DE LAS TECNOLOGÍAS DE LA INFORMACIÓN

Puesta en Producción Segura

TEMA 5

Implantación de sistemas seguros de despliegado de software

Alberto Diéguez Álvarez

Tabla de contenido

1.- Descripción de la tarea.....	3
Caso práctico.....	3
Apartado 1: Manejo básico de contenedores con Docker	3
Apartado 2: Dockerfile	13
Apartado 3: Docker-compose	18

1.- Descripción de la tarea.

Caso práctico



[dotCloud, Inc.](#), [Docker Logo](#) ([Apache License 2.0](#))

[dotCloud, Inc.](#), [Docker Logo](#) ([Apache License 2.0](#))

Julián se ha unido a un grupo de trabajo para la creación de un aplicativo web para entornos web, sabe que sus compañeros están usando Docker y aún tiene cierto temor al uso de esta tecnología por desconocimiento por lo que va repasando cada paso que da.

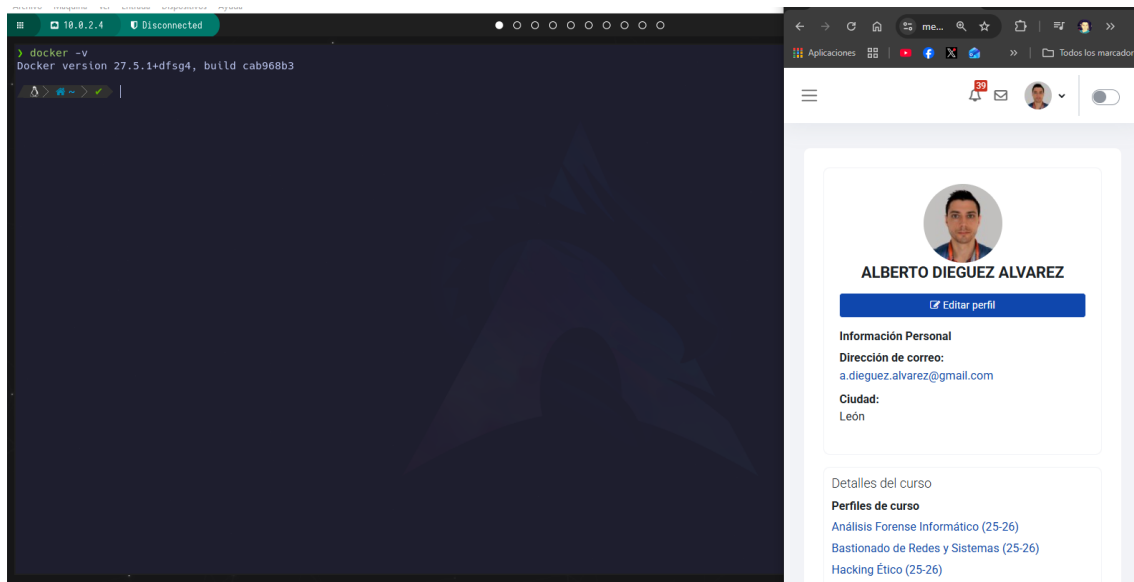
¿Qué te pedimos que hagas?

Esta actividad tiene parte teórica y parte práctica.

Apartado 1: Manejo básico de contenedores con Docker

1. En las unidades 2 y 3 ya se ha utilizado Docker así que ya debes tenerlo instalado, en caso contrario debes instalarlo. Puedes instalarlo en tu equipo o crear una máquina virtual e instalarlo en la misma (en ese caso te recomendamos utilizar una máquina Ubuntu porque va a ser muy sencillo).

Compruebo la instalación.



2. Rellena la siguiente tabla explicando que hacen las siguientes líneas

Comando/línea de comandos	Descripción detallada (si hay opciones se deben explicar)
<code>docker pull mysql:5.7.28</code>	Descarga la imagen de mysql:5.7.28 de Docker Hub
<code>docker images</code>	Lista todas las imágenes disponibles en el sistema
<code>docker run --rm --user mysql -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v ./mysql:/var/lib/mysql mysql:5.7.28</code> (NOTA: si el equipo es windows se debe cambiar ./mysql por .\mysql)	➤ Crea y ejecuta un contenedor. ➤ Con --rm se elimina el contenedor al pararlo. ➤ Con --user mysql se ejecuta como usuario mysql ➤ Con -d se ejecuta en segundo plano ➤ Con -p 33060:3306 mapea el puerto 33060 del host del 3306 del contenedor ➤ Con --name mysql-db1 le da nombre al contenedor ➤ Con -e MYSQL_ROOT_PASSWORD=secret define la contraseña para root ➤ Con -v ./mysql:/var/lib/mysql crea un volumen para persistir datos entre el contenedor y el host
<code>docker ps -a</code>	Se listan todos los contenedores tanto activos como parados
<code>docker logs mysql-db1</code>	Muestra los logs del contenedor mysql-db1
<code>docker exec -it mysql-db1 /bin/sh</code>	Crea una terminal interactiva dentro del contenedor
<code>docker stop mysql-db1</code>	Detiene el contenedor mysql-db1

Comando/línea de comandos	Descripción detallada (si hay opciones se deben explicar)
<code>docker start mysql-db1</code>	Arranca el contenedor mysql-db1
<code>docker rm -f mysql-db1</code>	Elimina el contenedor mysql-db1, lo hace un modo force, que significa que aunque este ejecución, lo detiene antes de borrarlo

3. Realizar pruebas con Jenkins utilizando contenedores.

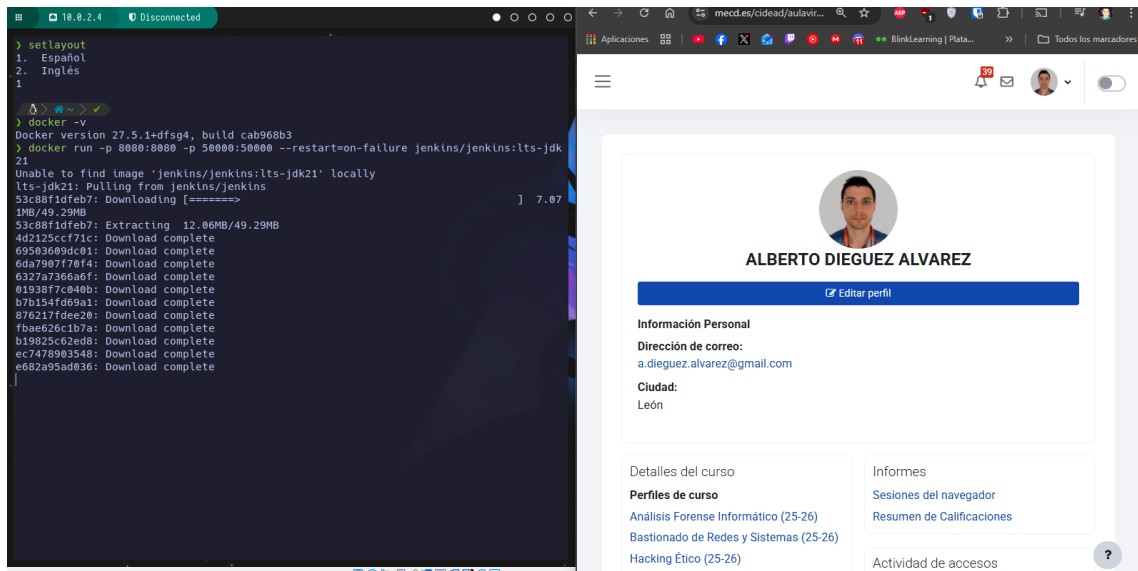
Jenkins es una herramienta de integración continua y entrega continua (CI/CD) de código abierto, escrita en Java, que permite automatizar el desarrollo, prueba y despliegue de software. Visita su página oficial para entenderlo mejor: <https://www.jenkins.io/>

Incluye pantallazos de las pruebas que se piden a continuación, incluyendo las de la consola donde se lanzan las líneas de comandos docker y del navegador con la aplicación.

3.1 Lanzar el contenedor de Jenkins (ver <https://github.com/jenkinsci/docker/blob/master/README.md>). Indica la línea de comandos para realizar esta operación.

Leo el README y proceso a ejecutar el comando:

```
docker run -p 8080:8080 -p 50000:50000 --restart=on-failure
jenkins/jenkins:lts-jdk21
```

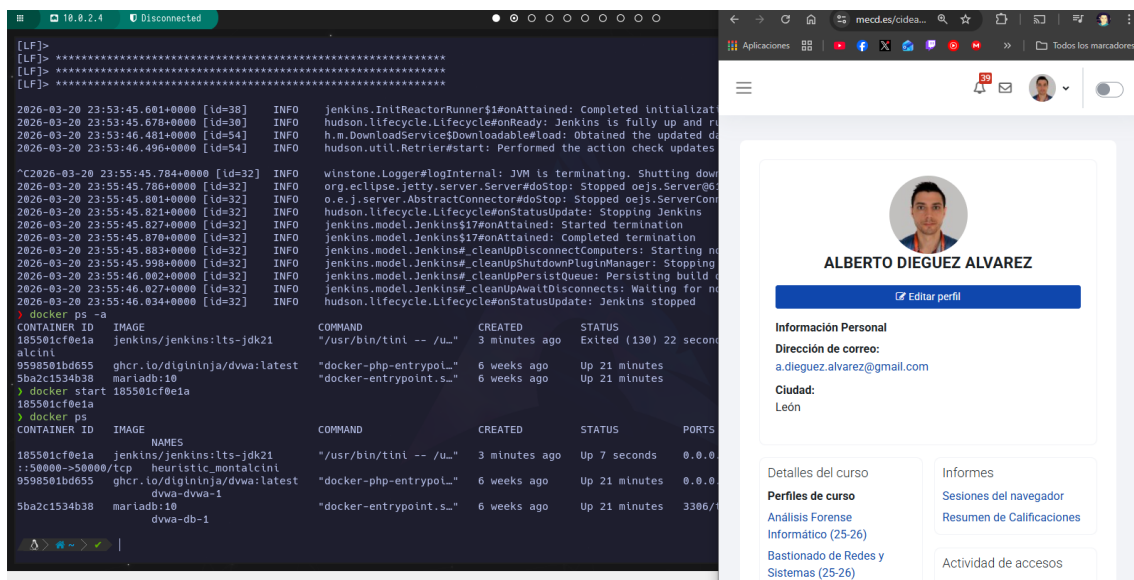


Procede a descargar la imagen primero y luego levantará el contenedor.

3.2 Comprobar que el contenedor anterior está funcionando. Indica la línea de comandos para realizar esta operación.

Ejecuto el siguiente comando:

`docker ps`



Lo paré y lo levanté con Docker start... podría haberlo ejecutado en modo detached.

Inserto el password que me aparece en los logs:

```
[LF]:
[LF]: Jenkins initial setup is required. An admin user has been created and a password generated.
[LF]: Please use the following password to proceed to installation:
[LF]:
[LF]: e91536bf157749afb33a3a0834a006c70
[LF]:
[LF]: This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]:
[LF]: *****
[LF]: *****
[LF]: *****
[2026-03-26 23:56:55.637+0000 [id=39] INFO jenkins.IntReactorRunner$1#onAttained: Completed initialization
[2026-03-26 23:56:55.917+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and ready to use
> docker ps
CONTAINER ID   IMAGE                                NAMES                                COMMAND                                CREATED        STATUS        PORTS
185591cf0e1a   jenkins/jenkins:lts-jdk21         "/usr/bin/tint -- /u..."          8 minutes ago   Up 4 minutes   0.0.0.0:
:50080->50080/tcp
heuristic-montalcini
9598516d6d55   ghcr.io/dwvwa/dwva:latest         "docker-php-entrypoin..."         6 weeks ago    Up 26 minutes   0.0.0.0:
80->80/tcp
dwva-dwva-1
5ba2c1534b38   mariadb:10                         "docker-entrypoint.s..."          6 weeks ago    Up 26 minutes   3306/
:3306/tcp
dwva-db-1
```



ALBERTO DIEGUEZ ALVAREZ


[Editar perfil](#)

Información Personal

Dirección de correo:
a.dieguez.alvarez@gmail.com

Ciudad:
León

Detalles del curso

Perfiles de curso

Análisis Forense
Informático (25-26)

Bastionado de Redes y

Informes

Sesiones del navegador

Resumen de Calificaciones

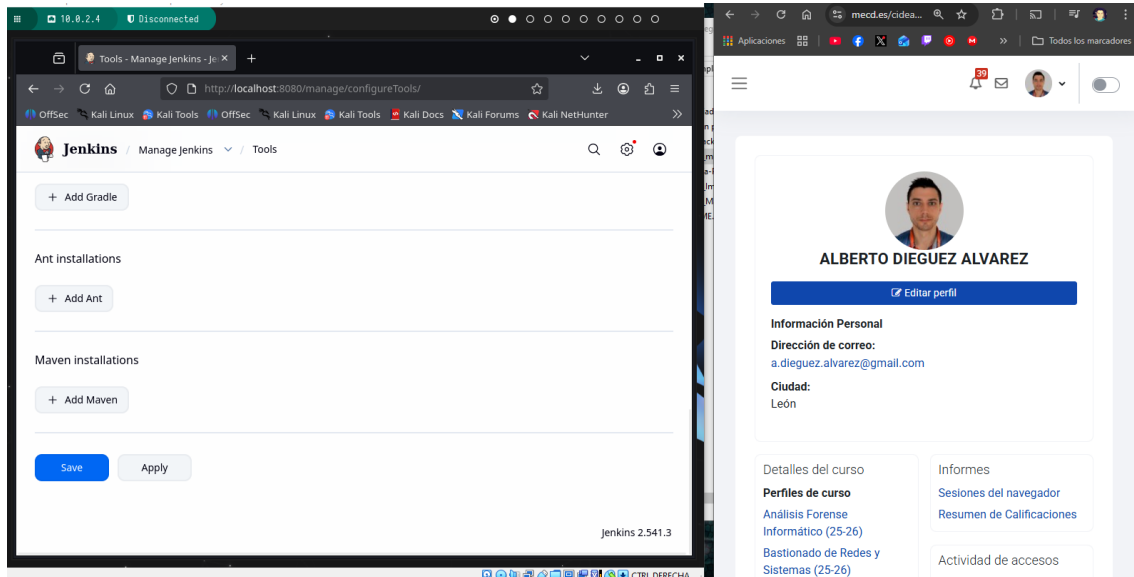
Actividad de accesos

Selecciono instalación sugerida:

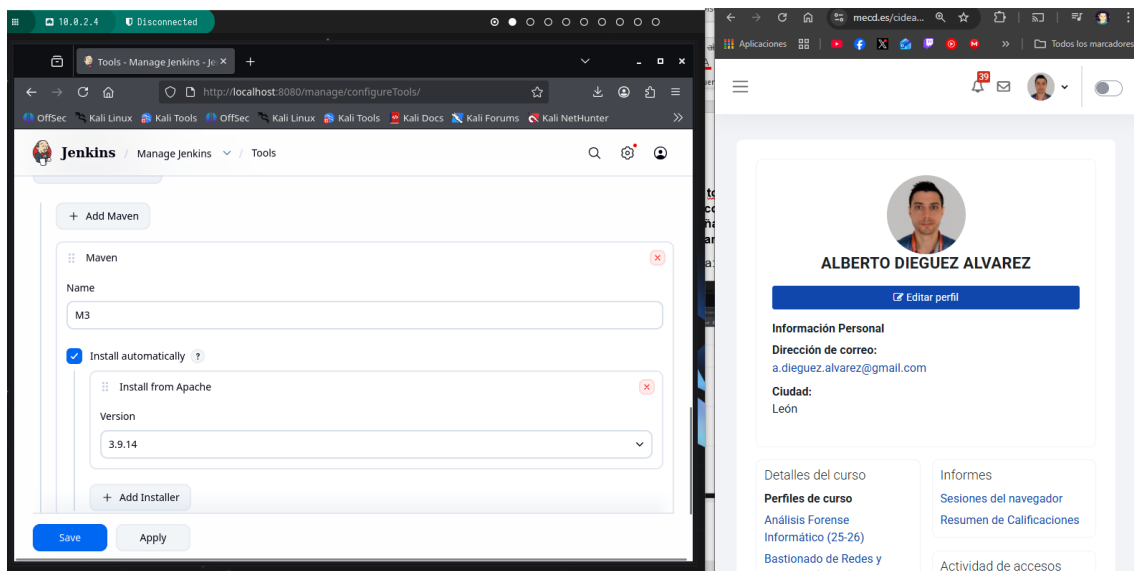
Termino de configurarlo:

3.5 Instalar la tool de Maven con el nombre M3 dentro de Jenkins (Panel de control - Administrar Jenkins - Tools - Instalaciones de Maven - Añadir Maven - Nombre M3 y la opción de instalar automáticamente)

Voy a donde se indica:

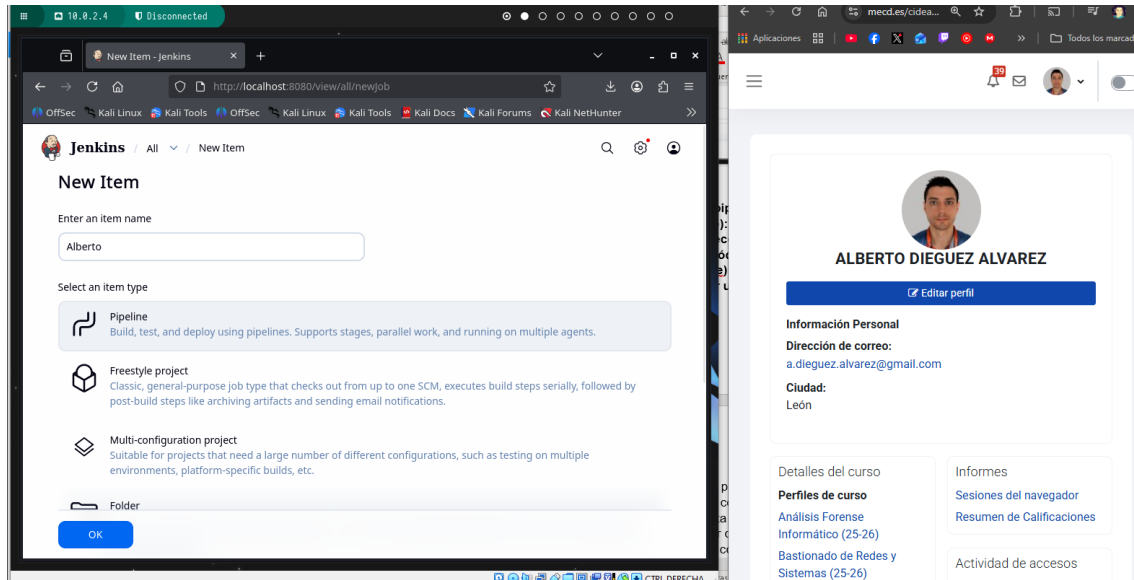


Le pongo nombre y acepto:

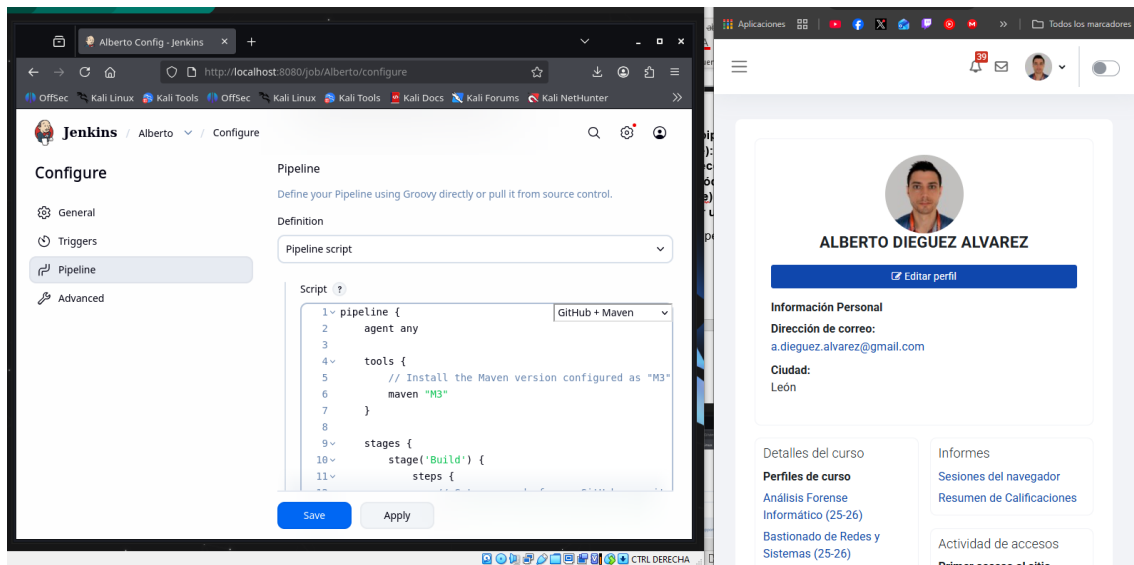


3.6 Crear un pipeline sencillo que se llame XXXXX (donde XXXXX es tu nombre): creas una tarea de tipo pipeline y en el apartado script seleccionas try sample Pipeline... escoges Github+ Maven. Copia el código del pipeline (a ese archivo se le denomina Jenkinsfile) en algún sitio porque te va a hacer falta para responder una pregunta en un apartado posterior

Procedo a crear el Pipeline:

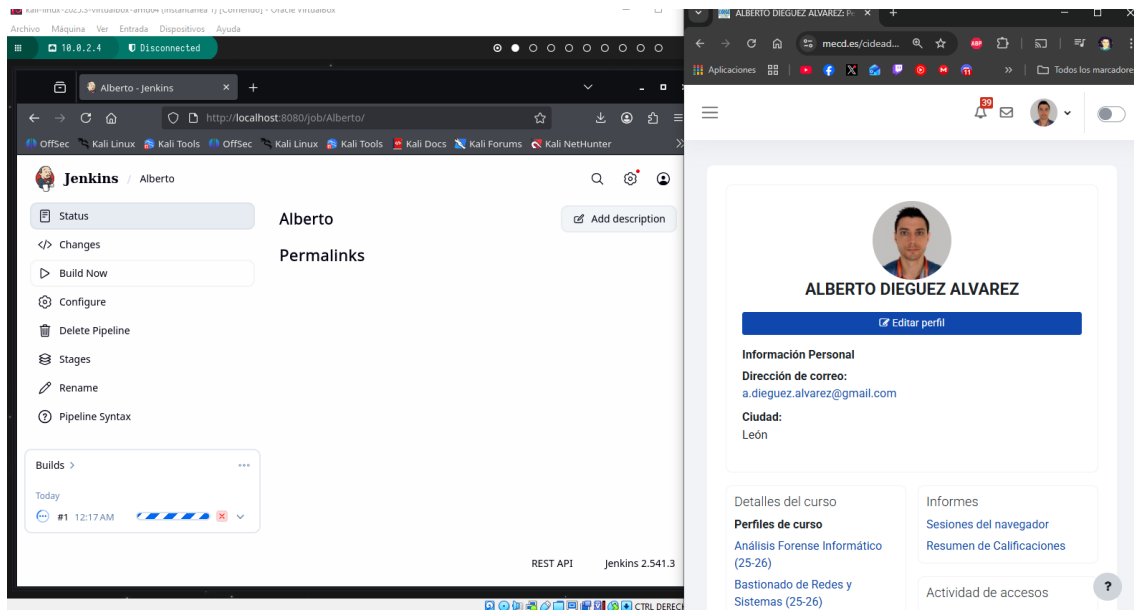


Configuro el Pipeline como se pide:

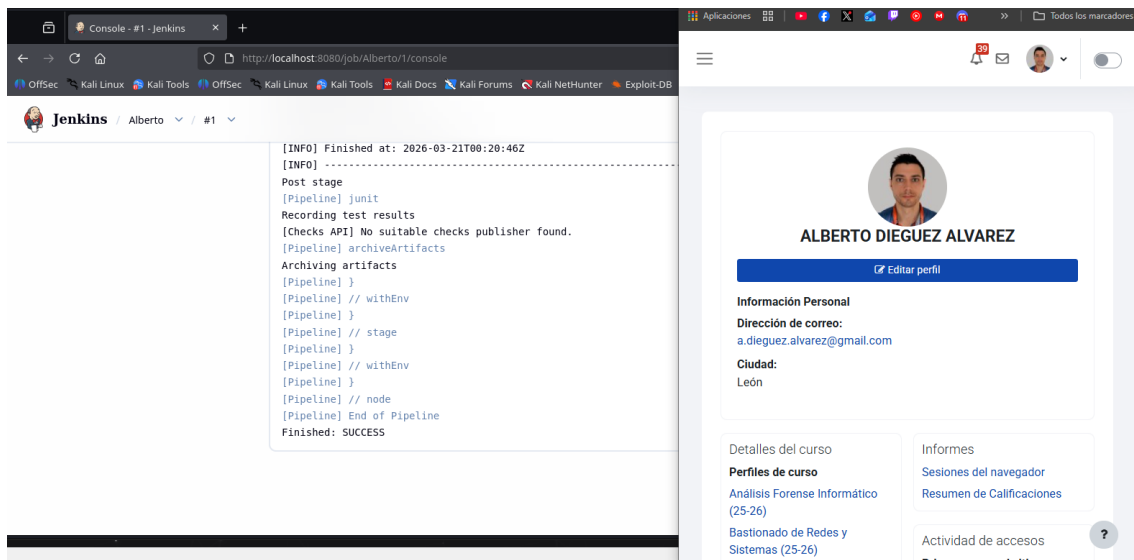


3.7 Ejecutar el pipeline anterior y mostrar el pantallazo de la salida.

Ejecuto el Pipeline en Build Now:



En console output veo lo que está haciendo:

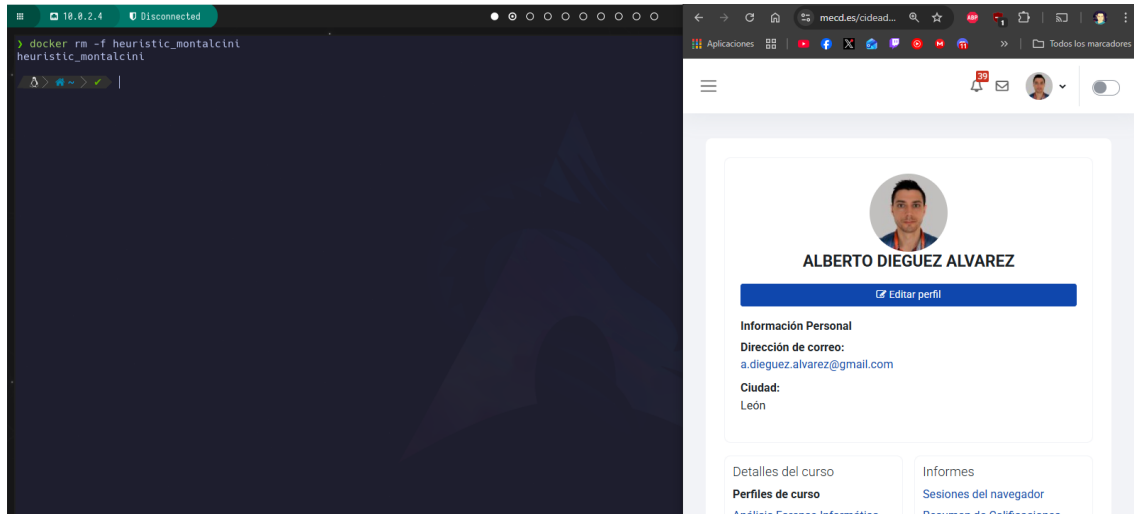


Se puede ver el full.log, pero en ese pantallazo justo ya se ve el final.

3.8 Eliminar el contenedor de Jenkins. Indica la línea de comandos para realizar esta operación.

Ejecuto el siguiente comando:

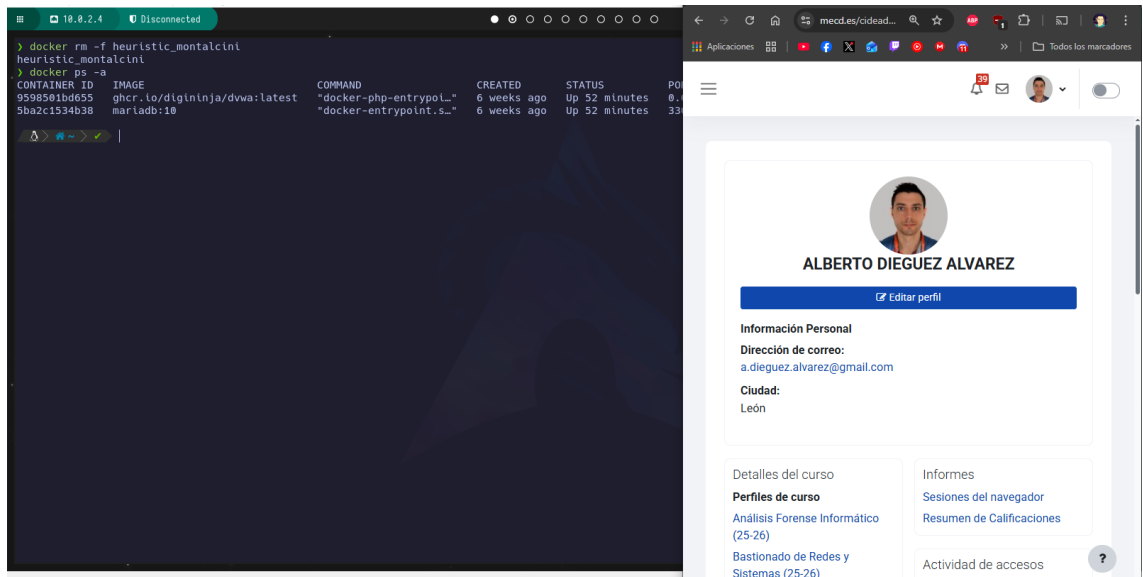
```
docker rm -f heuristic_montalcini
```



3.9 Comprobar que el contenedor anterior ya no está funcionando. Indica la línea de comandos para realizar esta operación.

```
docker ps -a
```

Y el contenedor ya no está:



4. Responde a las siguientes preguntas

4.1 ¿Qué ventajas aporta trabajar con contenedores?

Entre muchas, por ejemplo, la portabilidad ya que funciona igual en cualquier entorno, también aísla ya que cada app va independiente. El despliegue es muy rápido y consume pocos recursos.

4.2 Revisa el pipeline que has creado en el apartado anterior ¿Qué hace?

Primero usa cualquier agente disponible (agent any). Luego configura Maven (M3) con la versión que nos indicaba. Hace un git clone de un repositorio. Luego ejecuta mvn clean package con los tests. Por último, nos da un reporte y almacena el jar compilado.

4.3 Busca información sobre pruebas de seguridad en un pipeline e indica que pasos se podrían añadir al pipeline anterior para asegurar que la aplicación que se va a poner en producción es segura

Pues se podrían añadir nuevos stages con comprobaciones como por ejemplo:

- Análisis de dependencias
- Análisis de código
- Escaneo de contraseñas en texto claro
- Ejecutar un escaneo de vulnerabilidades

Apartado 2: Dockerfile

1. Dado el siguiente archivo Dockerfile

```
FROM ubuntu
LABEL maintainer="admin@ejemplo.com"
ARG APP_DATO1=1
RUN apt-get update && apt-get install -y apache2
RUN mkdir /var/www/html/ejemplo
EXPOSE 80
ADD index2.html /var/www/html/
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

Rellena la siguiente tabla donde tendrás que indicar **para que sirve cada línea**

FROM ubuntu	Es la imagen base sobre la que se construye el contenedor.
LABEL maintainer="admin@ejemplo.com"	Simplemente añade más información, en este caso el mantenedor
ARG APP_DATO1=1	Es una variable de construcción, y solo permanece durante el build
RUN apt-get update && apt-get install -y apache2	Actualiza los repositorios e instala apache2
EXPOSE 80	El contenedor usará el puerto 80
ADD index2.html /var/www/html/	Copia el archivo index2.html a la ruta que le indica dentro del contenedor
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]	Es el comando principal que se ejecuta al iniciar el contenedor, en este caso arranca apache en primer plano

2. Responde a las siguientes preguntas y realiza las pruebas

2.1. ¿Para qué es útil un archivo Dockerfile?

Sirve para definir como será una imagen Docker, automatiza la instalación del software y configura el entorno.

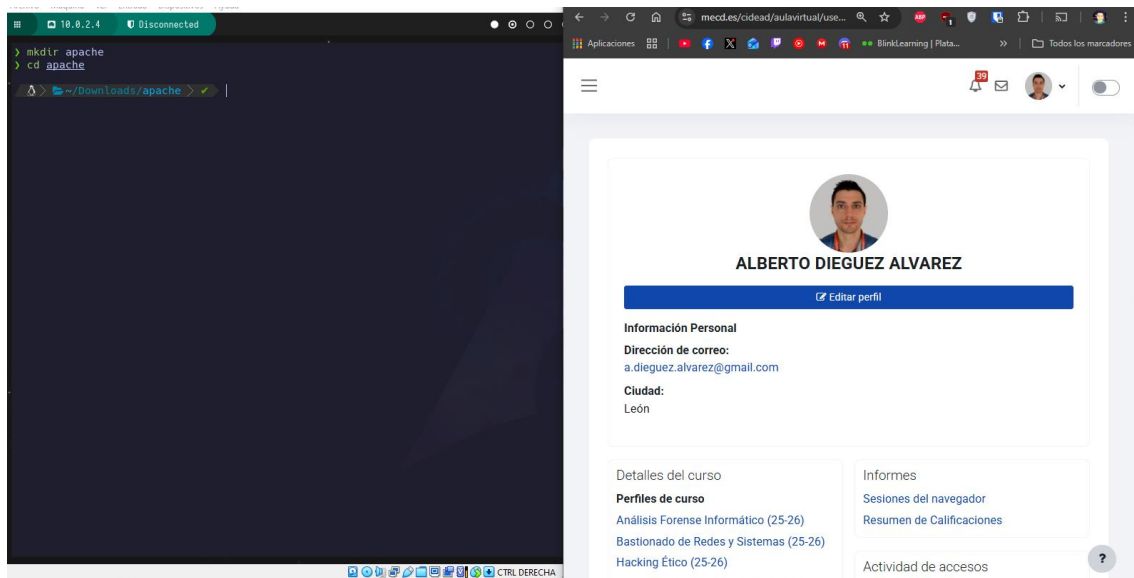
2.2. ¿Con qué comando se crean las imágenes partiendo de un Dockerfile?

```
docker build -t nombre_imagen:versión .
```

2.3. Realiza una prueba con el archivo anterior:

2.3.1. Crea un directorio

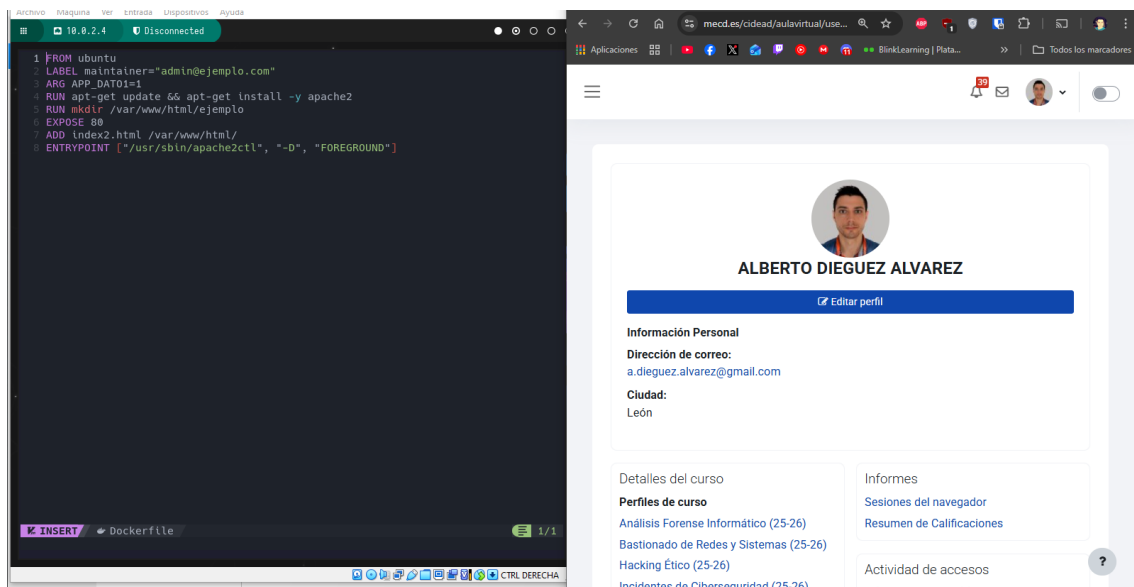
```
mkdir apache  
cd apache
```



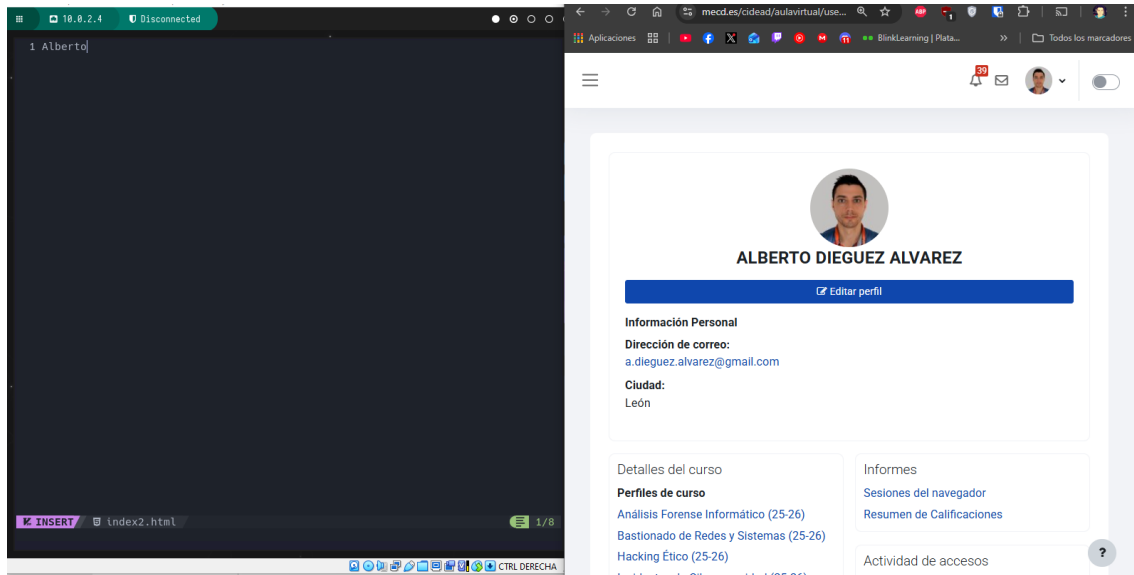
2.3.2. Crea dentro del directorio el archivo Dockerfile anterior

Ejecuto: vi Dockerfile

Y dentro pego el código anterior.



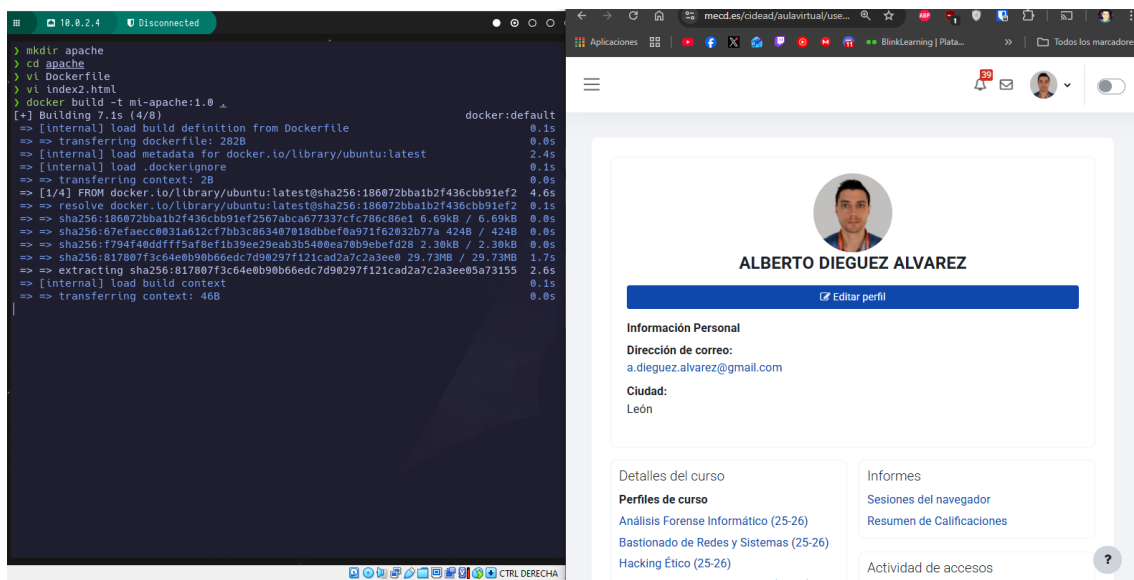
2.3.3. Crea el archivo index2.html (escribe dentro tu nombre)



2.3.4. Crea una imagen nueva con ese archivo Dockerfile de nombre mi-apache y versión 1.0

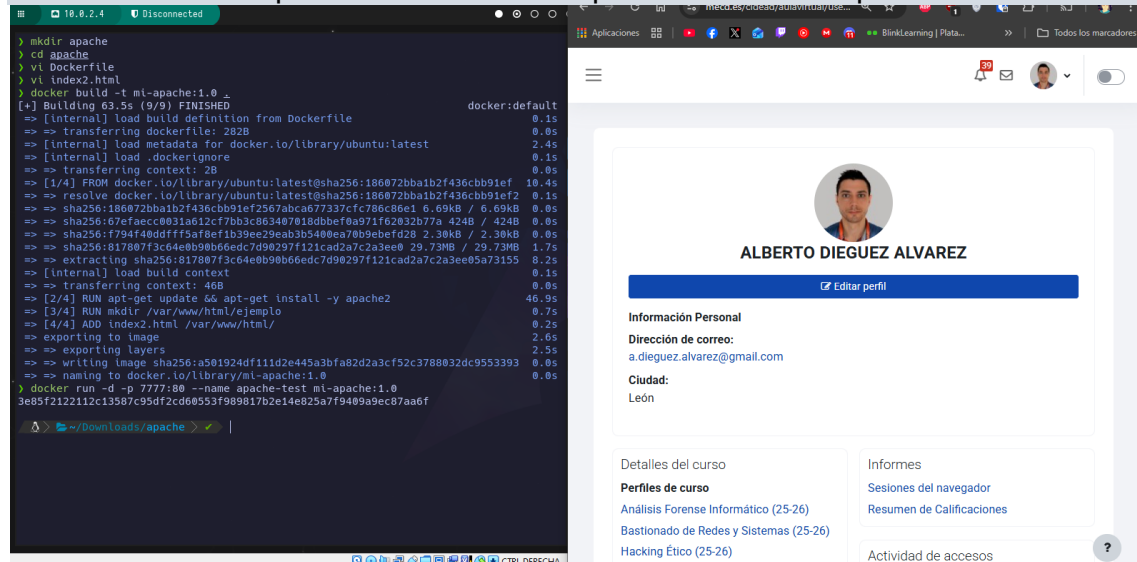
Ejecuto el siguiente comando:

```
docker build -t mi-apache:1.0 .
```

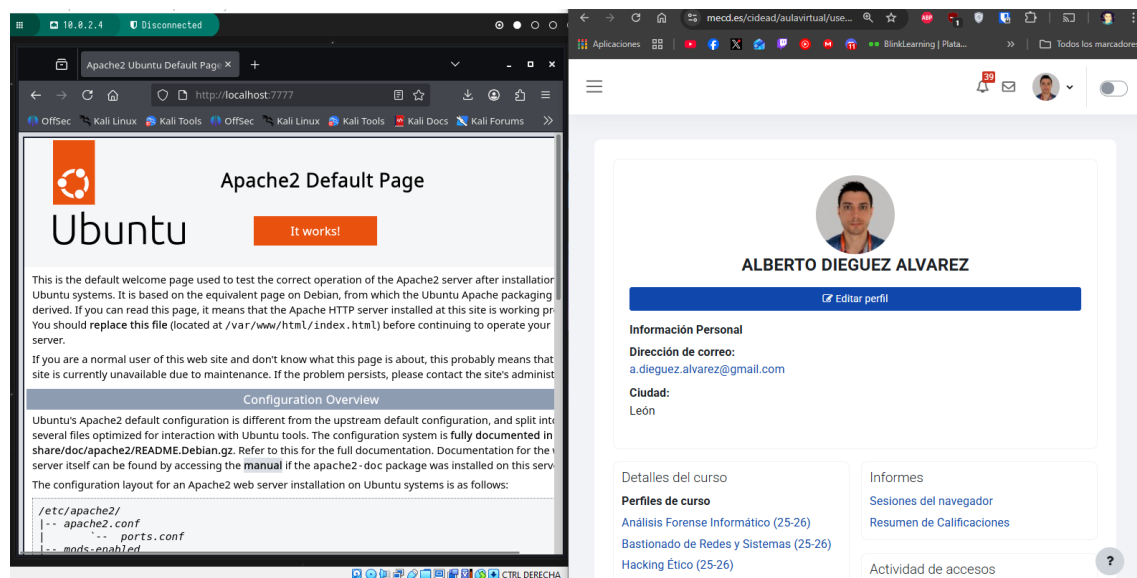


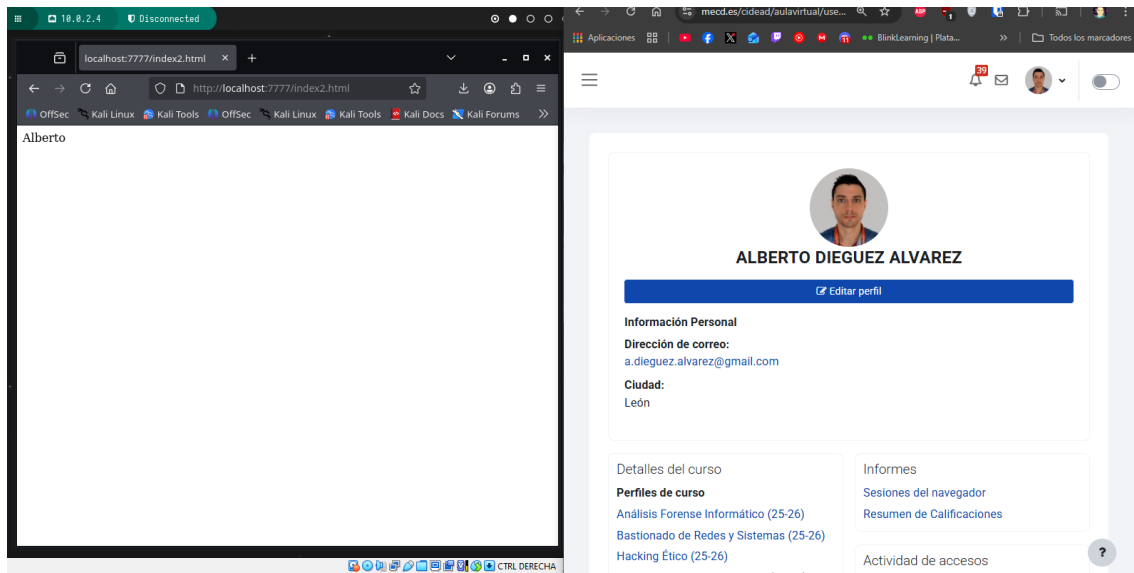
2.3.5. Lanza un contenedor con la imagen anterior en el puerto 7777.

```
docker run -d -p 7777:80 --name apache-test mi-apache:1.0
```



2.3.6. Abre un navegador y accede al contenedor anterior mostrando las páginas index.html e index2.html





Apartado 3: Docker-compose

1. **Rellena la siguiente tabla** indicando para que sirve cada línea de comandos u opción dentro de un archivo docker-compose.yml

línea de comandos: docker-compose up	Levanta todos los servicios que están definidos en el docker-compose.yml
línea de comandos: docker-compose down	Elimina los contenedores, redes... que han sido creados con docker-compose up
services	Donde se definen los servicios (contenedores) que forman parte de la aplicación
image	Nos indica la imagen que se usará para el contenedor
build	Indica la ruta del Dockerfile para construir la imagen
container-name	Nombre del contenedor
ports	Mapea los puertos entre el host y en contenedor host:contenedor
volumes	Monta carpetas dentro del contenedor para crear persistencia

environment

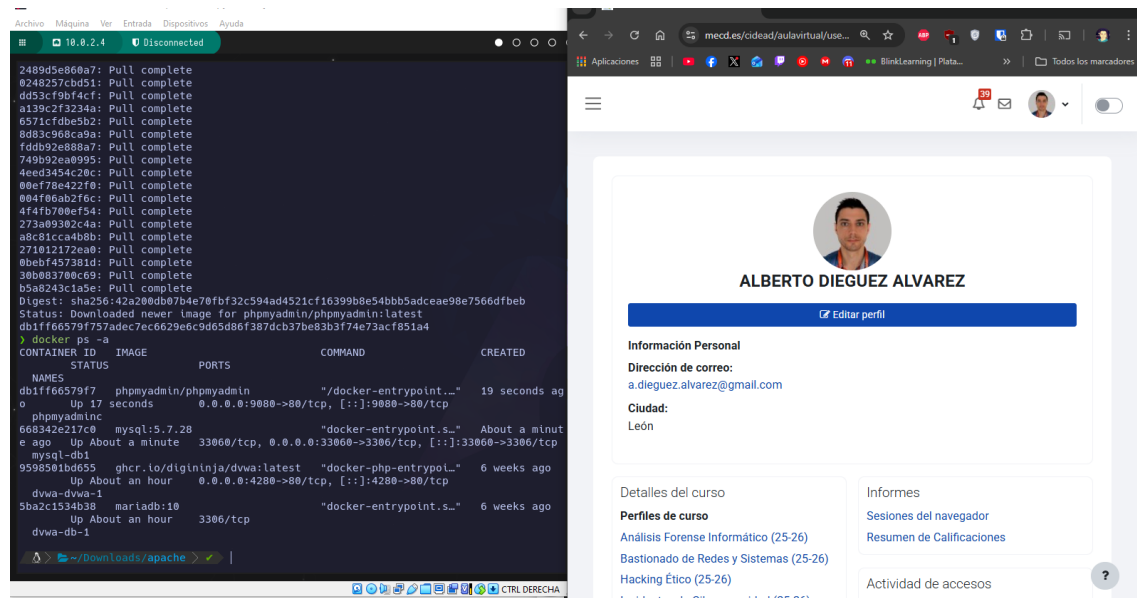
Define las variables de entorno dentro del contenedor

2. Realiza las siguientes pruebas (incluye pantallazos de cada una)

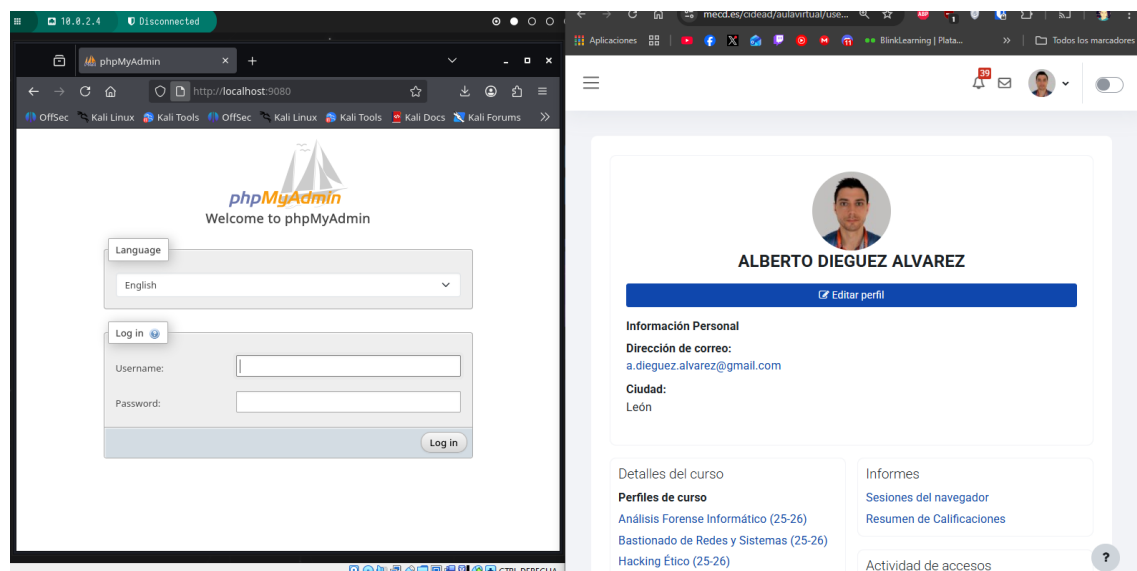
Lanza dos contenedores con las siguientes líneas en una terminal y **muestra pantallazos de la ejecución de esos comandos (docker ps -a)**

```
docker run -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v mysql:/var/lib/mysql mysql:5.7.28
```

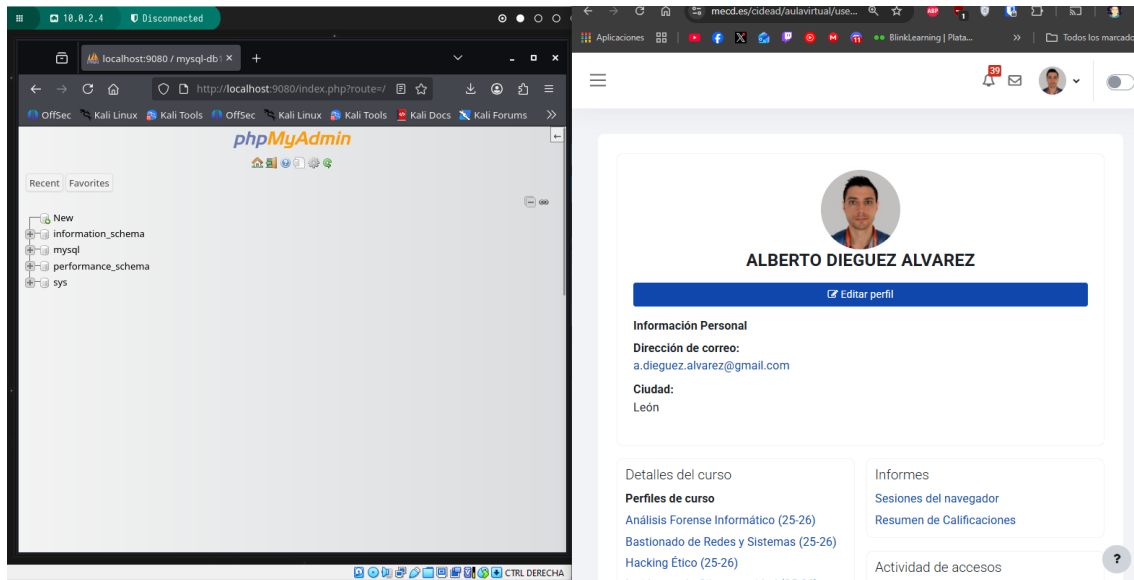
```
docker run -d --rm --name phpmyadminc --link mysql-db1 -e PMA_HOST=mysql-db1 -p 9080:80 phpmyadmin/phpmyadmin
```



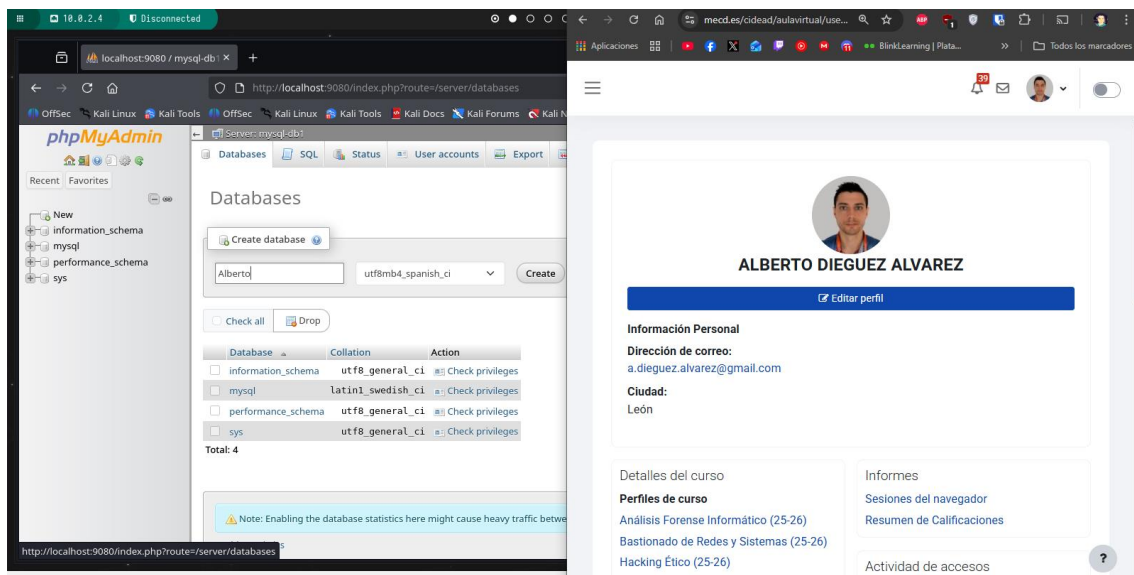
2.1. Utilizando el contenedor de phpmyadmin crea una base de datos con tu nombre en mysql. Muestra pantallazos del navegador con el acceso a phpmyadmin y con la creación de la base de datos
Entro en phpMyAdmin:



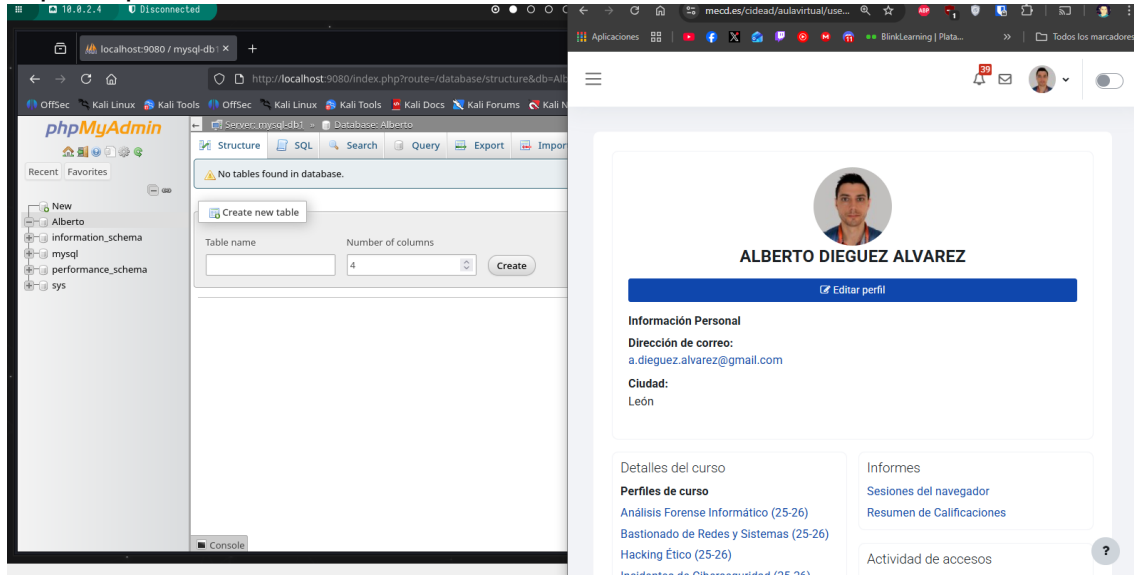
Con el user: root , y la pass: secret, accedo al index.



En settings y en Databases, configuro y creo la base de datos con mi nombre:

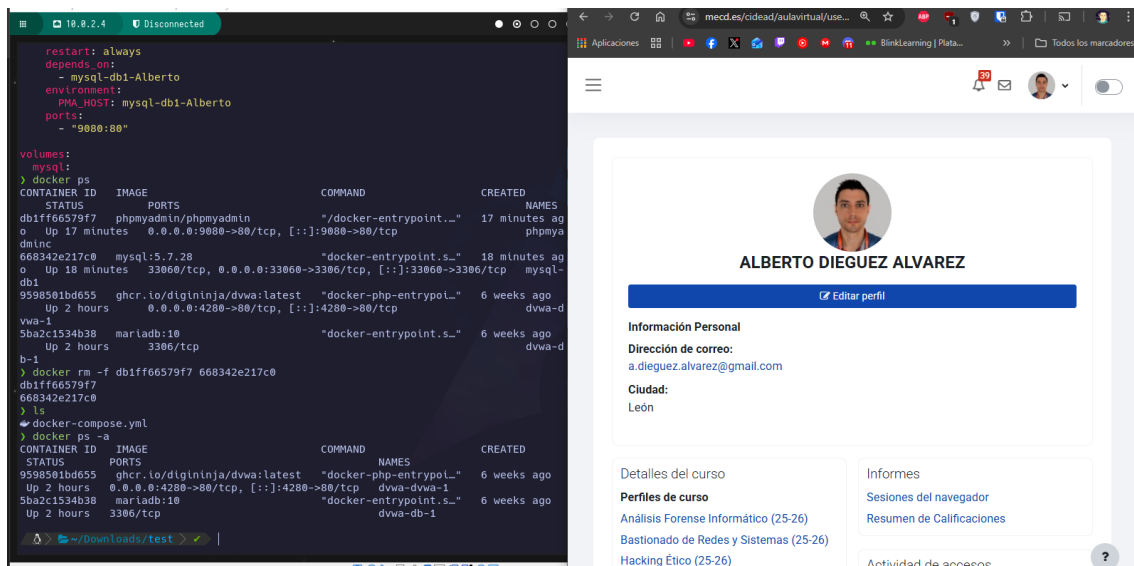


Aquí se puede ver la BBDD con mi nombre creada:

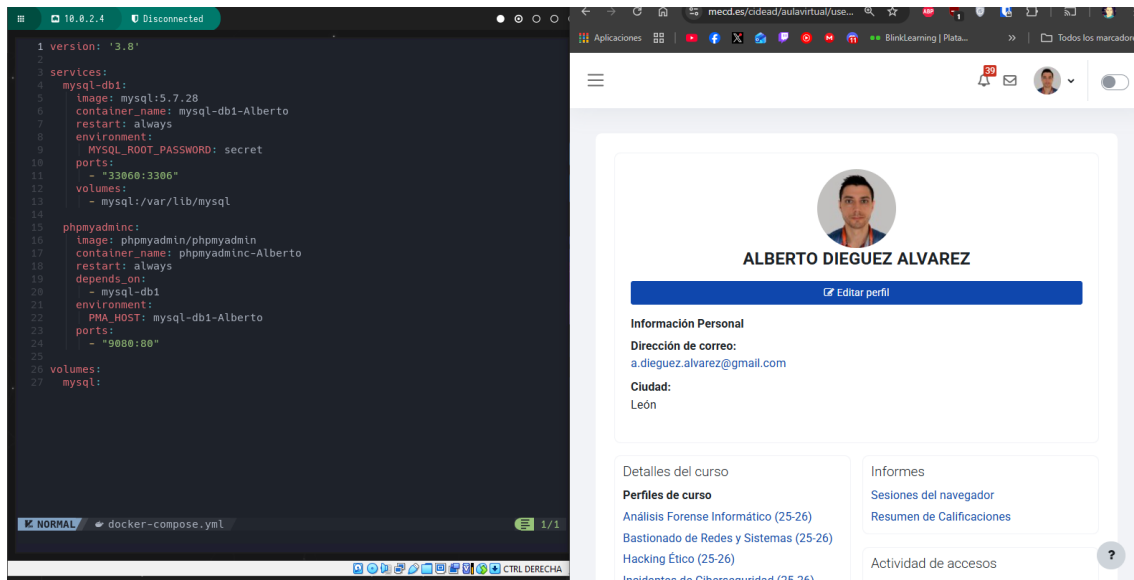


2.2. Crea un archivo docker-compose.yml que permita lanzar esos dos contenedores utilizando docker compose. Incluye el contenido del archivo docker-compose.yml

Para tener los puertos libres, borro los contenedores anteriores:



Siguiendo la estructura de una de las preguntas anteriores, creo el Docker compose:



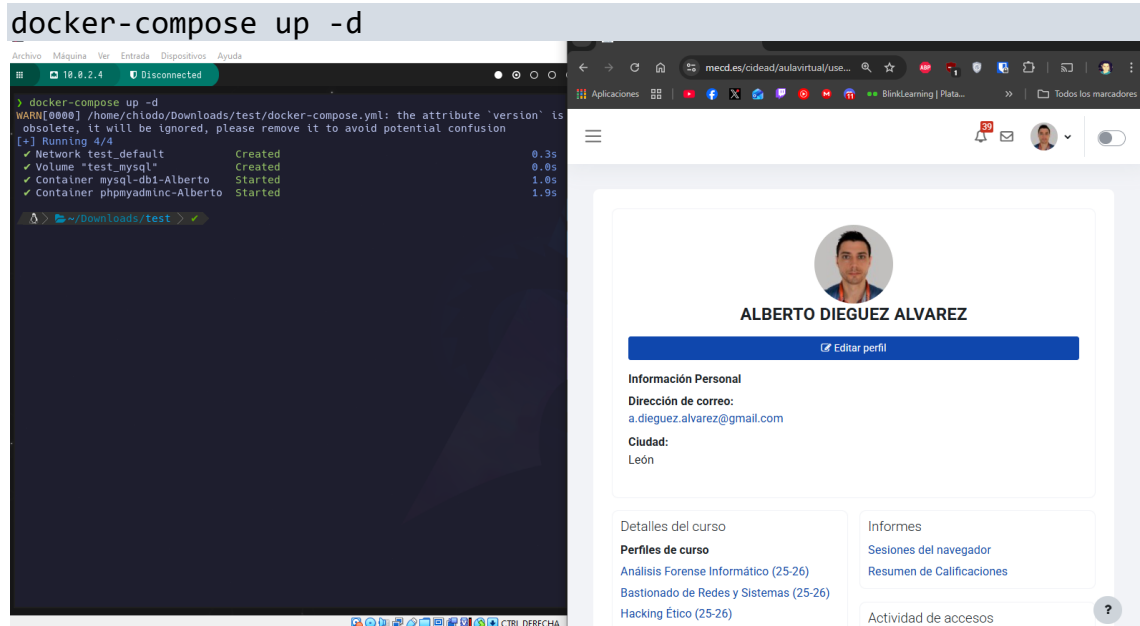
```
version: '3.8'

services:
  mysql-db1:
    image: mysql:5.7.28
    container_name: mysql-db1-Alberto
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: secret
    ports:
      - "33060:3306"
    volumes:
      - mysql:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: phpmyadminc-Alberto
    restart: always
    depends_on:
      - mysql-db1
    environment:
      PMA_HOST: mysql-db1-Alberto
    ports:
      - "9080:80"

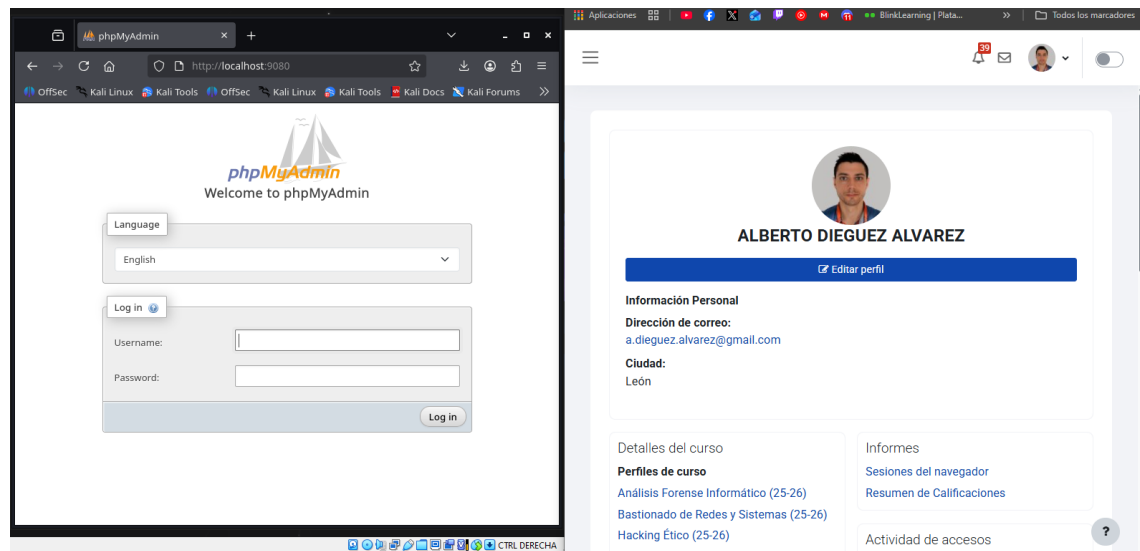
volumes:
  mysql:
```

2.3. Lanza el archivo anterior. Muestra pantallazos de la ejecución en la consola



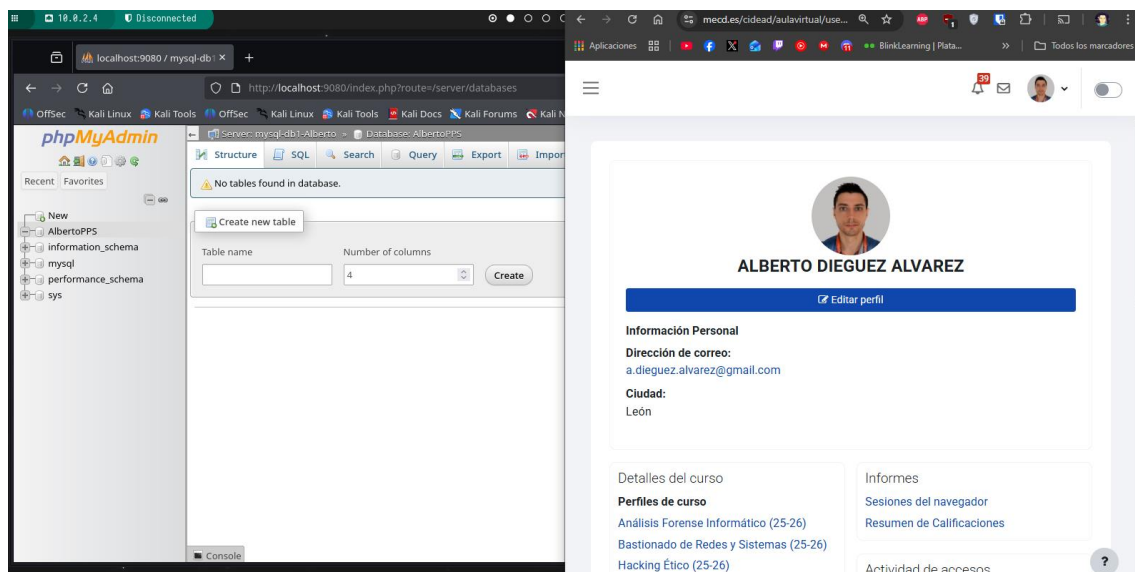
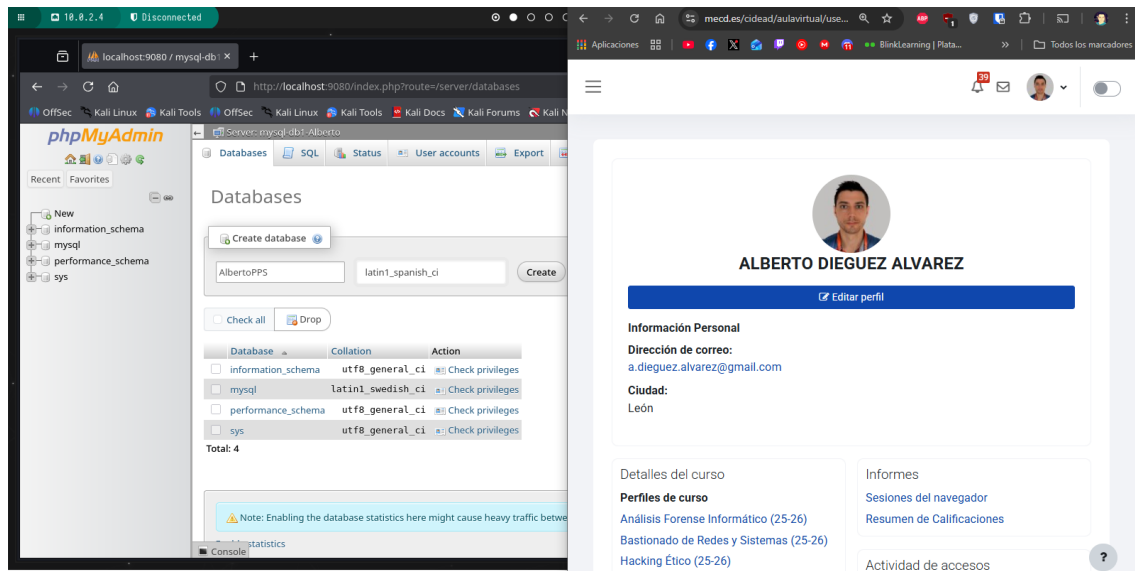
2.4. Utilizando el contenedor de phpmyadmin crea una tabla en la base de datos. Muestra pantallazos del navegador con el acceso a phpmyadmin y con la creación de la tabla

Entro a phpmyadmin:



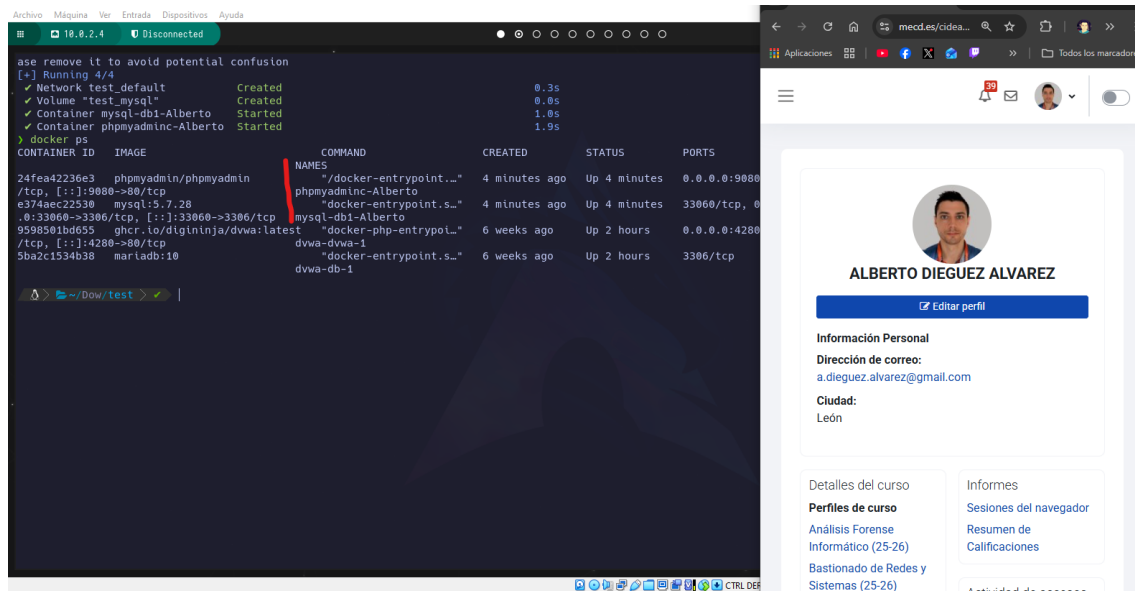
Lo único que se ve exactamente igual que lo anterior.

Creo una tabla como he hecho antes.



2.5. Termina los contenedores lanzados. Muestra pantallazos de la ejecución en la consola

Aquí se ven los contenedores creados con el docker-compose.yml.



Procedo a pararlos con docker-compose down:

